Lecture 9

- Video Streaming
- P2P Networking

Question [from previous exams]

Select those statements that are *true*.

a) The ***"NS record"*** in DNS maps a hostname to an IP address.

b) DNS caching happens ***only*** at the local DNS server.

c) The root DNS servers store the IP addresses for ***all*** hosts on the Internet.

d) In a ***recursive DNS query***, the DNS server contacted is responsible for resolving the name completely and returning the final answer to the client.

e) In DNS, ***"CNAME record"*** maps an alias hostname to the canonical hostname.

Question [from previous exams]

- Assume you are the founder of a new startup company, called *NeuralMesh.com*, which provides agentic AI solutions for mesh network configurations. The following are the information for web, email, and DNS servers for your company.
  - Web server: www.NeuralMesh.com has IP address: 129.237.120.20
  - Mail server: mail.NeuralMesh.com has IP address: 129.237.120.25
  - DNS server: dns1. NeuralMesh.com has IP address 129.237.120.10
- ***Part 1:*** List the resource records (RRs) that need to be provided to the .com TLD registrar. Provide (name, value, type) for each RR.

- ***Part 2:*** List the RRs that need to be added to the authoritative DNS server at your company.

Video Streaming and CDNs: context

- Stream video traffic: major consumer of Internet bandwidth







- ***Challenge 1***: **heterogeneity**
  - different users have different capabilities (e.g., wired versus mobile; bandwidth-rich versus bandwidth-poor)

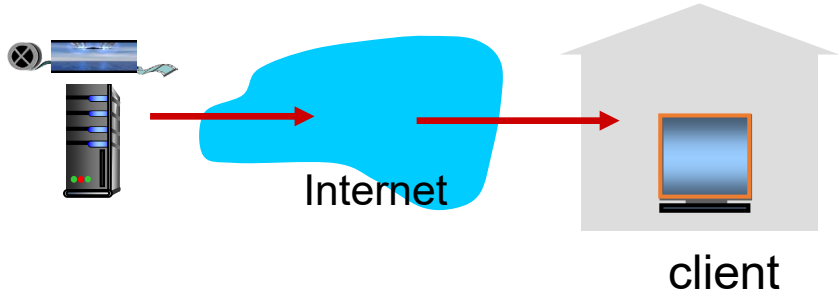


- ***Challenge 2***: **scale:** how to reach ~1B users?
  - single mega-video server won't work
- *Solution: distributed, application-level infrastructure*



Solution 1 - Streaming multimedia: DASH

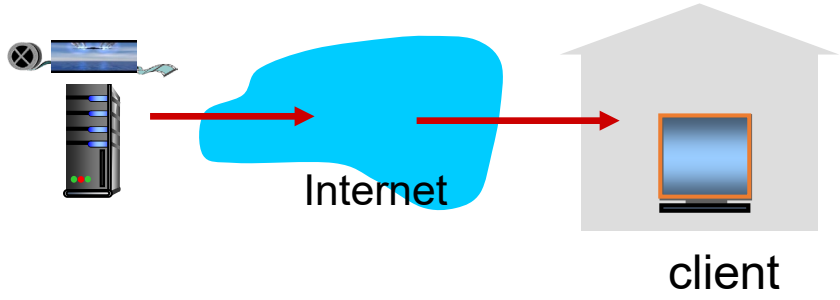
- *DASH: D*ynamic, *A*daptive *S*treaming over *H*TTP
- *server:*
  - divides video file into multiple chunks
  - each chunk stored, encoded at different rates
  - ***manifest file:*** provides URLs for different chunks



- *client:*
  - periodically measures server-to-client bandwidth
  - consulting manifest, requests one chunk at a time
    - chooses maximum encoding rate sustainable given current bandwidth
    - can choose different encoding rates at different points in time (depending on available bandwidth at time)

Solution 1 - Streaming multimedia: DASH

- *"intelligence"* at client: client determines
  - *when* to request chunk (so that buffer starvation, or overflow does not occur)
  - *what encoding rate* to request (higher quality when more bandwidth available)
  - *where* to request chunk (can request from URL server that is "close" to client or has high available bandwidth)



Streaming video = encoding + DASH + playout buffering

Solution 2 - Content distribution networks (CDNs)

- *challenge:* how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- option 1: single, large "mega-server"
  - single point of failure
  - point of network congestion
  - long path to distant clients
  - multiple copies of video sent over outgoing link

➔ this solution *doesn't scale*

Content distribution networks (CDNs)

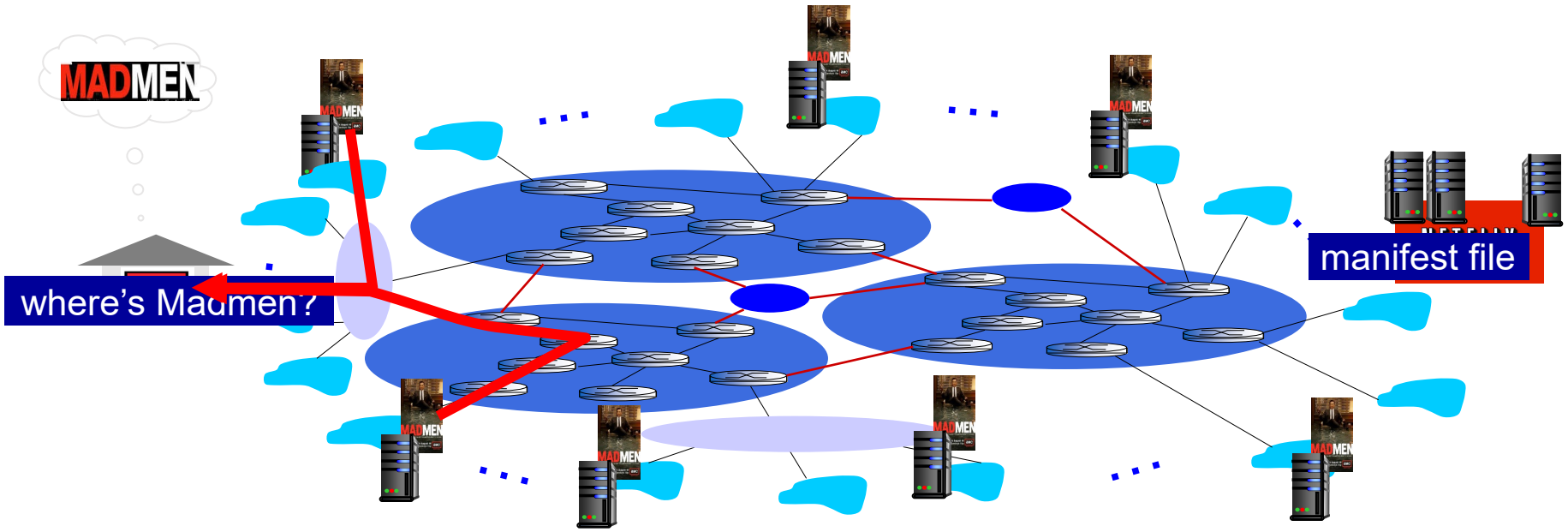
- *challenge:* how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?
- option 2: store/serve multiple copies of videos at multiple geographically distributed sites *(CDN)*
  - *"enter deep":* push CDN servers deep into many access networks ➔ close to users; Example: Akamai: 350,000 servers deployed in more than 130 countries
  - *"bring home":* smaller number (10's) of larger clusters in point of presence (POPs) near (but not within) access networks. Example: Fastly





Content distribution networks (CDNs)

- CDN: stores copies of content at CDN nodes
  - e.g. Akamai stores copies of MadMen
- subscriber requests content from CDN
  - directed to nearby copy, retrieves content
  - may choose different copy if network path congested



Content distribution networks (CDNs)



Internet host-host communication as a service

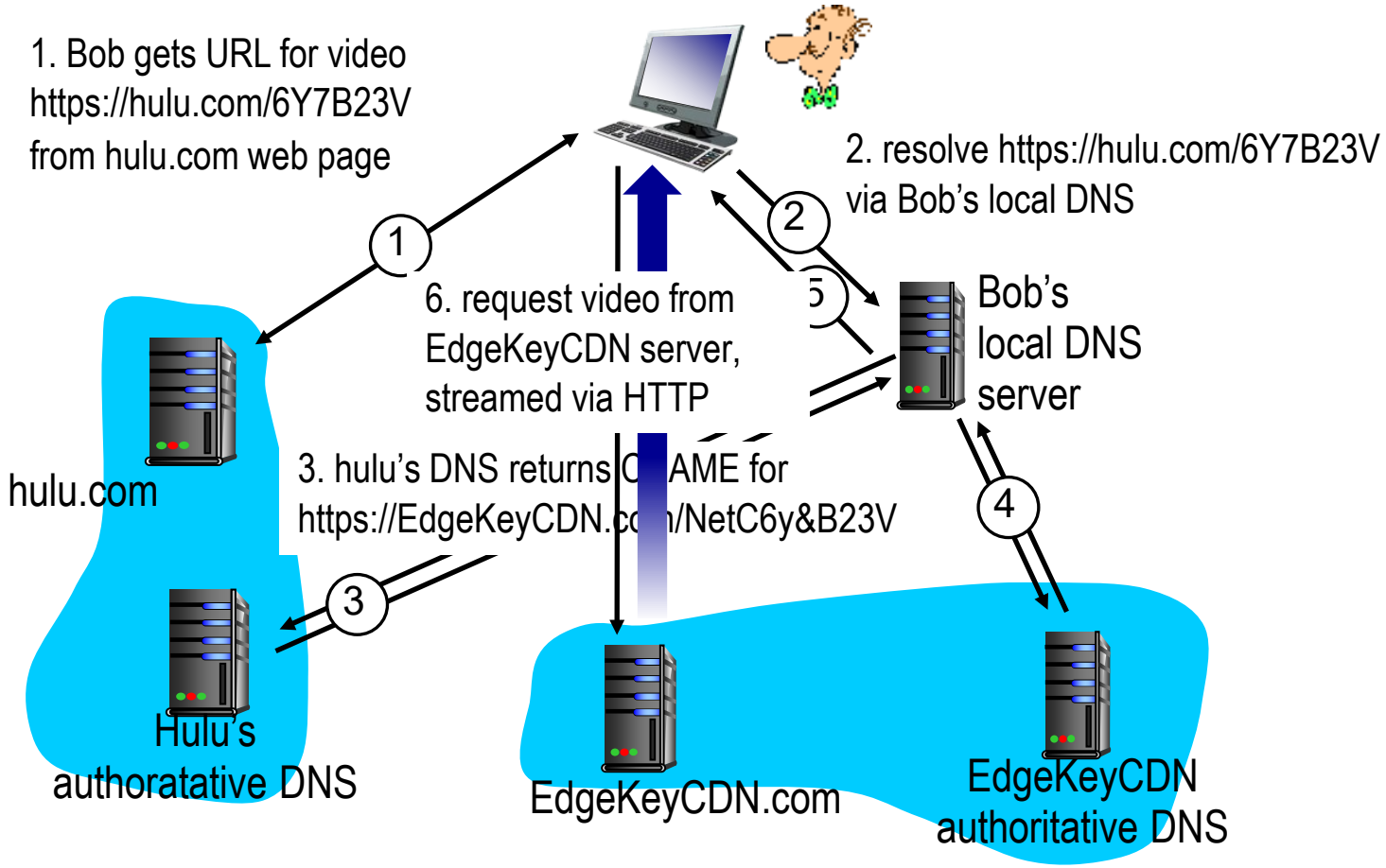
OTT challenges: coping with a congested Internet

- from which CDN node to retrieve content?
- viewer behavior in presence of congestion?
- what content to place in which CDN node?

CDN content access: a closer look

Bob (client) requests video https://hulu.com/6Y7B23V

- video stored in CDN at https://EdgeKeyCDN.com/NetC6y&B23V



Real-world example

- Akamai EdgeKey Servers



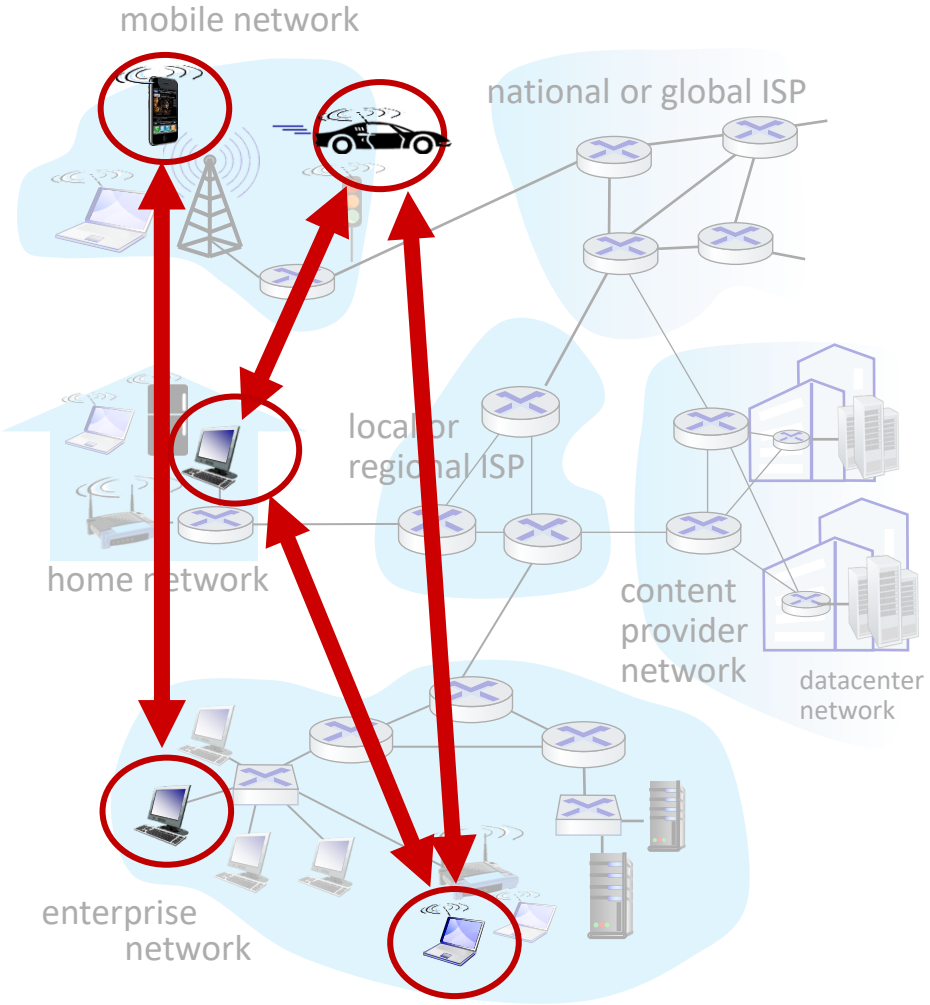

```
┌──(kali㉿kali)-[~]
└─$ nslookup -type=cname www.hulu.com
Server:         192.168.189.2
Address:        192.168.189.2#53

Non-authoritative answer:
www.hulu.com    canonical name = wildcard.hulu.com.edgekey.net.
```

P2P Networking
&
Socket Programming

Peer-to-peer (P2P) architecture

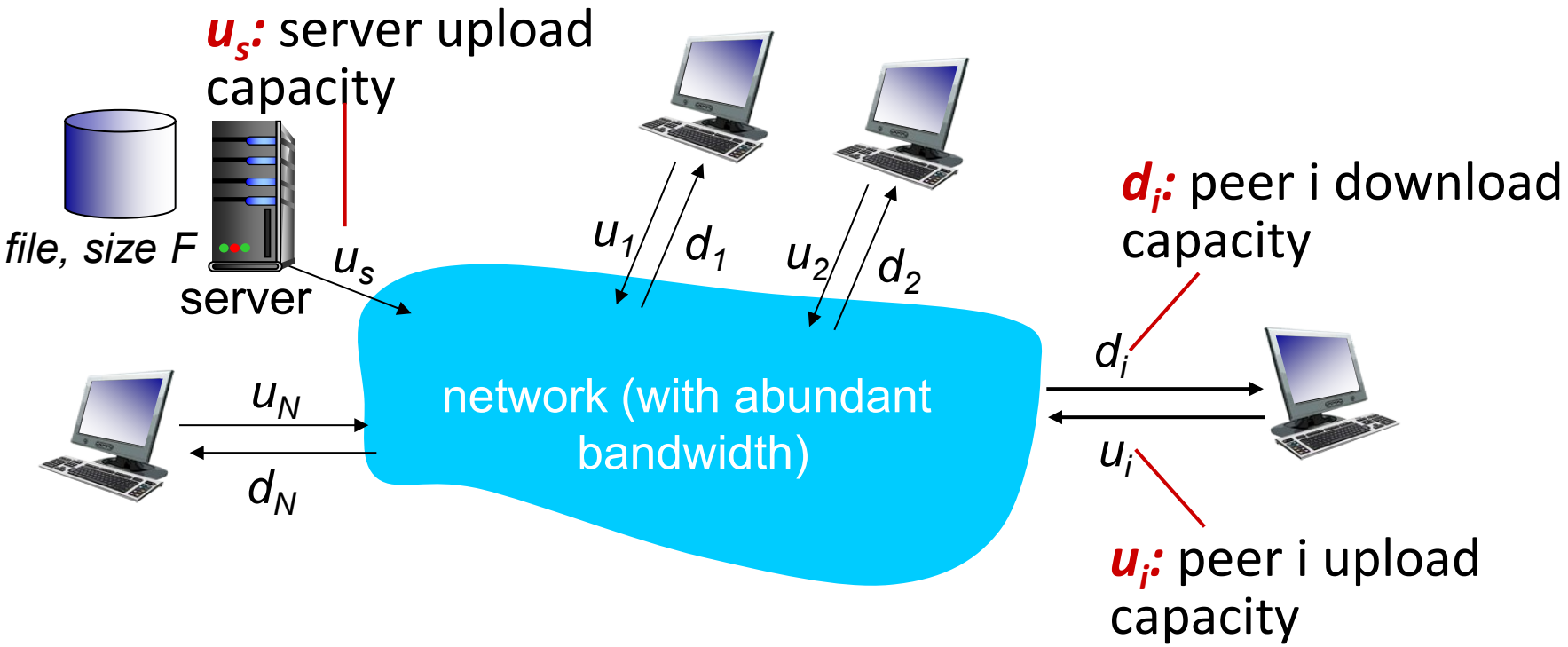
- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- examples: P2P file sharing (BitTorrent)



File distribution: client-server vs P2P

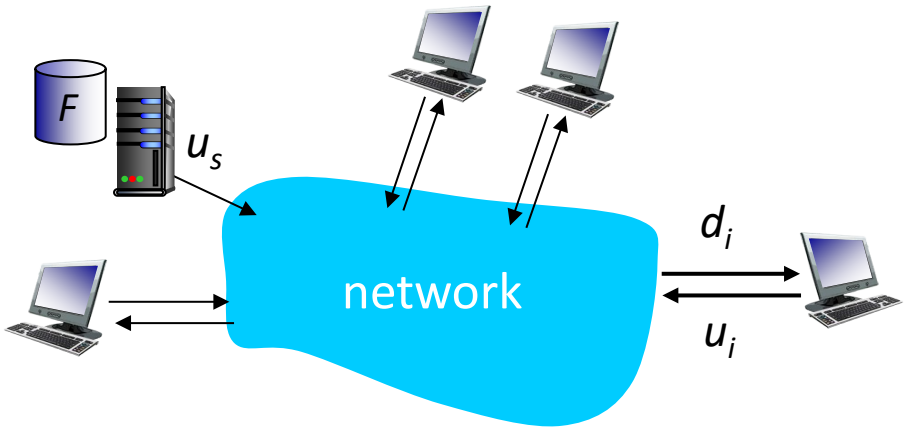
Q: how much time to distribute file (size *F*) from one server to *N peers*?

- peer upload/download capacity is limited resource



File distribution time: client-server

- *server transmission:* must sequentially send (upload) *N* file copies:
  - time to send one copy: F/u_s
  - time to send *N* copies: NF/u_s
- *client:* each client must download file copy
  - d_{min} = min client download rate
  - max client download time: F/d_{min}



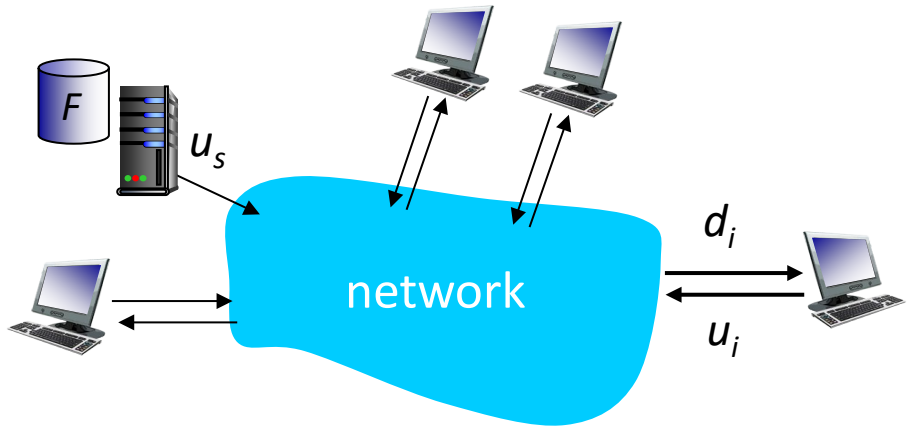
time to distribute F to N clients using client-server approach

$$D_{c\text{-}s} \geq max\{NF/u_s, F/d_{min}\}$$

increases linearly in N

File distribution time: P2P

- *server transmission:* must upload at least one copy:
  - time to send one copy: F/u_s
- *client:* each client must download file copy
  - max client download time: F/d_{min}
- *clients:* as aggregate must download NF bits
  - max upload rate (limiting max download rate) is $u_s + \Sigma u_i$



time to distribute F to N clients using P2P approach

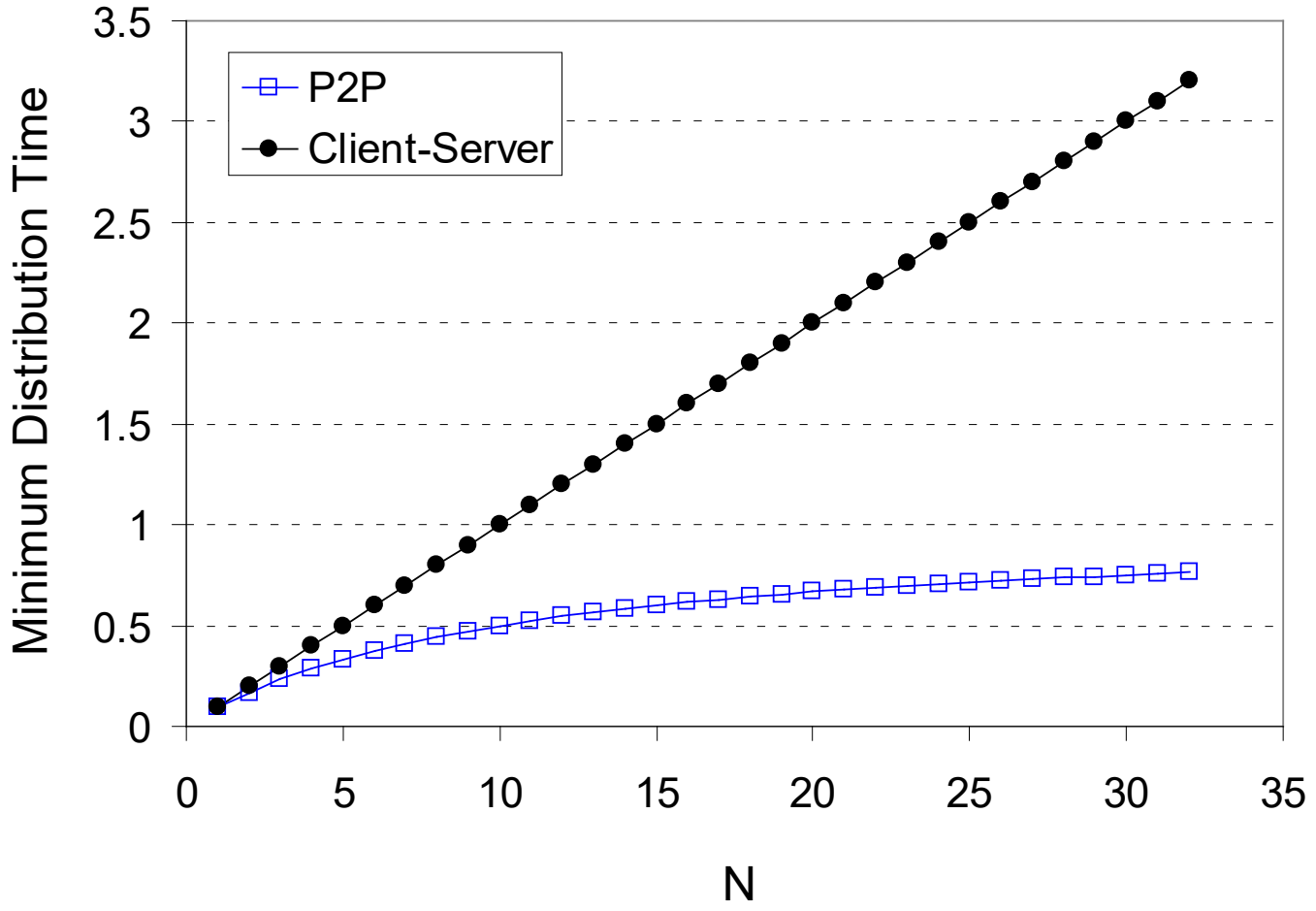
$$D_{P2P} \geq max\{F/u_s, F/d_{min}, NF/(u_s + \Sigma u_i)\}$$

increases linearly in N …

… but so does this, as each peer brings service capacity

Client-server vs. P2P: example

client upload rate = u, F/u = 1 hour, $u_s = 10u$, $d_{min} \geq u_s$

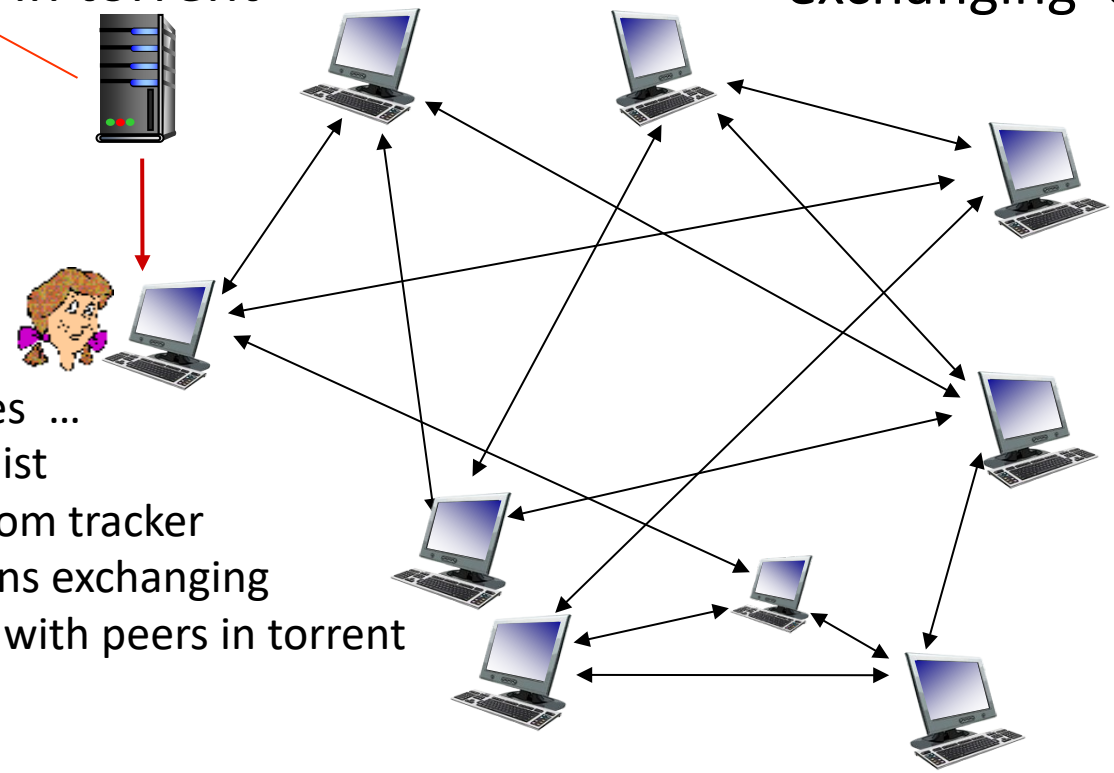


P2P file distribution: BitTorrent

- file divided into chunks
- peers in torrent send/receive file chunks

tracker: tracks peers participating in torrent

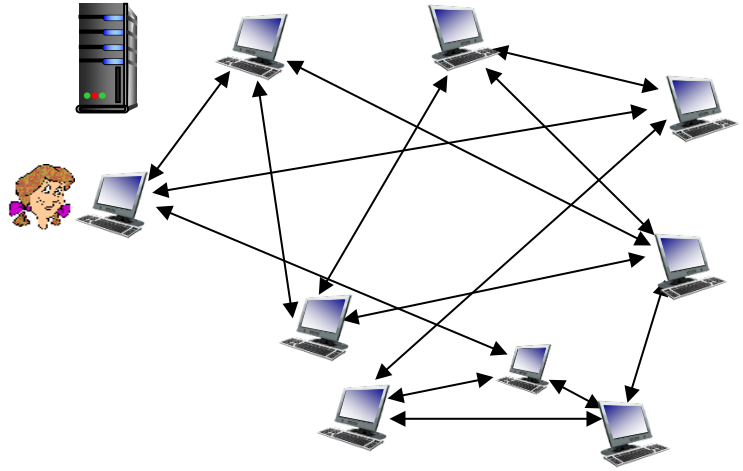
torrent: group of peers exchanging chunks of a file



Alice arrives …
… obtains list
of peers from tracker
… and begins exchanging
file chunks with peers in torrent

P2P file distribution: BitTorrent

- peer joining torrent:
  - has no chunks, but will accumulate them over time from other peers
  - registers with tracker to get list of peers, connects to subset of peers ("neighbors")



- while downloading, peer uploads chunks to other peers
- peer may change peers with whom it exchanges chunks
- *churn:* peers may come and go
- once peer has entire file, it may (selfishly) leave or (altruistically) remain in torrent

BitTorrent: requesting, sending file chunks

Requesting chunks:

- at any given time, different peers have different subsets of file chunks
- Alice requests missing chunks from peers, rarest first

Sending chunks: tit-for-tat

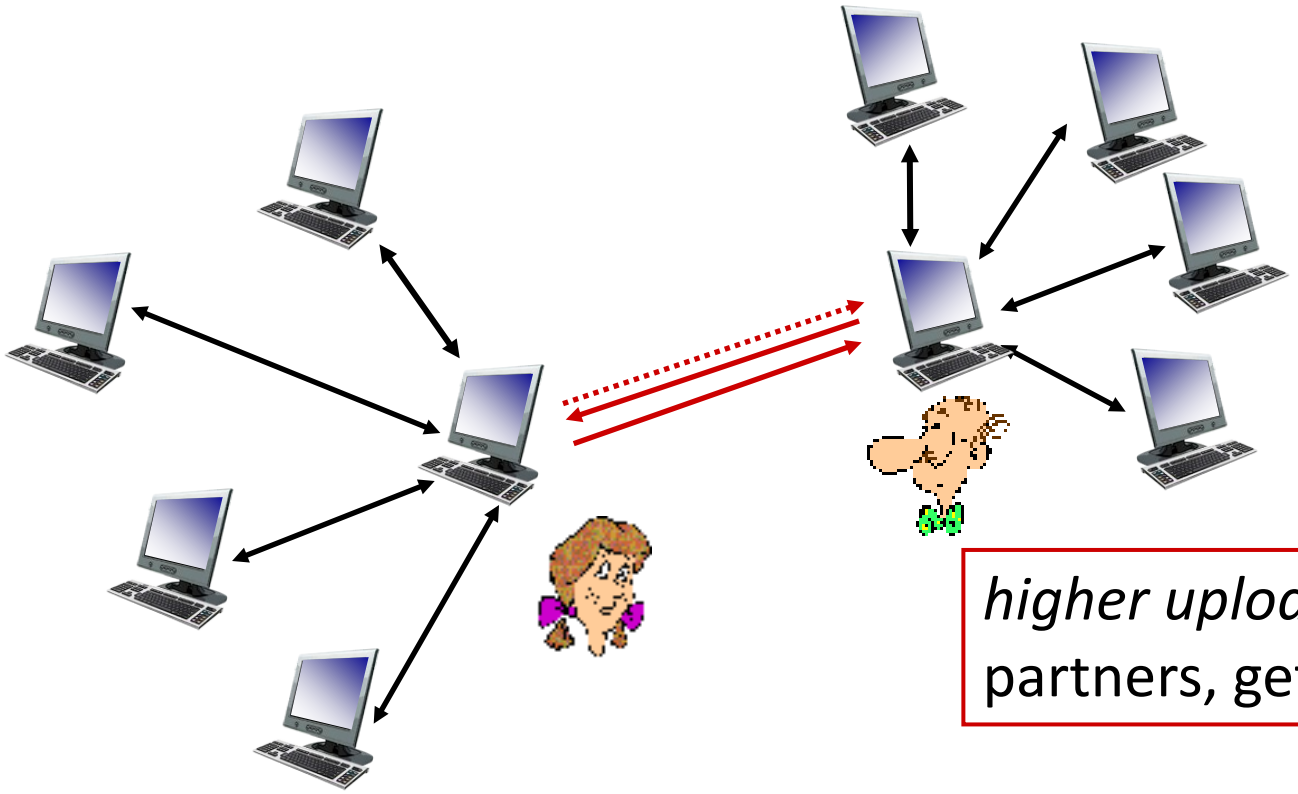
- Alice sends chunks to those four peers currently sending her chunks *at highest rate*
  - other peers are choked by Alice (do not receive chunks from her)
  - re-evaluate top 4 every few seconds
- every few seconds: randomly select another peer, starts sending chunks
  - "**optimistically unchoke**" this peer
  - newly chosen peer may join top 4

BitTorrent: tit-for-tat

(1) Alice “optimistically unchokes” Bob

(2) Alice becomes one of Bob’s top-four providers; Bob reciprocates

(3) Bob becomes one of Alice’s top-four providers



higher upload rate: find better trading partners, get file faster !